Echo-Mind

Advanced AI Voice Assistant

Project Documentation

Desktop Application built with Python & PyQt6

B.Tech Mini Project

Viharika | Echo-Mind

Table of Contents

1. Project Overview

Echo-Mind is a modern, AI-powered desktop voice assistant built with Python. It combines speech recognition, large language model (LLM) reasoning, text-to-speech synthesis, and system automation into a single, user-friendly desktop application.

The assistant listens for voice commands through the microphone, converts speech to text using Google's Speech Recognition API, processes commands through an intelligent routing system, and responds both visually (in the chat transcript) and audibly (via pyttsx3 text-to-speech with a female voice).

Key Features

- Voice-first interaction with continuous listening mode
- LLM-powered conversational AI (Groq / OpenAI integration)
- Text-to-Speech with female voice output and stop control
- YouTube search and direct video playback
- Google search integration
- Screenshot capture with automatic timestamped filenames
- Application launching (Chrome, Notepad, Calculator, VS Code, etc.)
- Wikipedia topic lookup with smart alias resolution
- Conversation memory with JSON persistence (up to 20 messages)
- Modern dark-themed PyQt6 desktop GUI with glassmorphism design
- Real-time animated waveform visualization
- Settings dialog for API key management

2. System Architecture

Echo-Mind follows a modular, layered architecture with clear separation of concerns. The application is divided into four main layers:

Presentation Layer (ui/)

The PyQt6-based desktop GUI provides the visual interface. It includes a two-panel layout with the main conversation area on the left and a control panel on the right. The GUI communicates with the backend via Qt signals and slots, ensuring thread-safe UI updates.

Controller Layer (modules/assistant_core.py)

The AssistantController acts as the central orchestrator. It manages the listening loop, dispatches commands to the task router or LLM, triggers text-to-speech, and emits signals to update the UI. All blocking operations run on background threads to keep the UI responsive.

Service Layer (modules/)

Individual service modules handle specific responsibilities: speech-to-text conversion, text-to-speech synthesis, LLM API communication, command routing, system automation, and web tasks. Each service is implemented as a class or set of functions with a singleton pattern.

Data Layer (utils/, memory/, config/)

Configuration is loaded from environment variables (.env file). Conversation history is persisted as JSON in the memory/ directory. Utility functions provide thread-safe file operations for both memory and environment management.

3. Project Structure

```
voice-assistant/
|-- main.py                     # Application entry point
|-- requirements.txt            # Python dependencies
|-- .env                        # Environment variables (API keys)
|-- .gitignore                  # Git ignore rules
|-- README.md                   # Project documentation
|
|-- config/
|   |-- config.py               # Configuration loader
|
|-- modules/
|   |-- assistant_core.py       # Central controller
|   |-- llm_engine.py           # LLM API integration
|   |-- speech_to_text.py       # Speech recognition
|   |-- text_to_speech.py       # TTS service manager
|   |-- tts_worker.py           # TTS subprocess worker
|   |-- task_router.py          # Command pattern matcher
|   |-- system_control.py       # OS-level automation
|   |-- web_tasks.py            # Wikipedia lookups
|
|-- ui/
|   |-- main_window.py          # PyQt6 desktop GUI
|
|-- utils/
|   |-- helpers.py              # Memory & env utilities
|
|-- memory/
|   |-- conversation_memory.json  # Chat history
|
|-- screenshots/                # Captured screenshots
```

4. Detailed File Descriptions

4.1 main.py - Application Entry Point

This is the entry point of the Echo-Mind application. It creates a PyQt6 QApplication instance, sets the application name and organization, instantiates the main window (EchoMindWindow), displays it, and starts the Qt event loop.

Key responsibilities:

- Initialize the Qt application framework
- Create and display the main window
- Run the event loop until the user closes the window

```
def main() -> int:
    app = QApplication(sys.argv)
    app.setApplicationName("Echo-Mind")
    window = EchoMindWindow()
    window.show()
    return app.exec()
```

4.2 config/config.py - Configuration Management

Manages all application configuration by loading values from environment variables defined in a .env file. Uses python-dotenv for loading. Every configurable parameter has a sensible default value so the app can start even without a .env file.

Configuration categories:

- Assistant identity: ASSISTANT_NAME (default: 'Echo-Mind'), USER_NAME
- Audio settings: VOICE_RATE (175 WPM), LISTEN_TIMEOUT (6s), LISTEN_PHRASE_LIMIT (8s)
- LLM settings: LLM_PROVIDER (groq/openai), API keys, model names, timeout
- Engine selection: STT_ENGINE (google), TTS_ENGINE (pyttsx3)
- Storage: MAX_MEMORY (20 messages), SCREENSHOT_DIR
- Debug: DEBUG_MODE flag for development logging

4.3 modules/assistant_core.py - Core Controller

The AssistantController is the heart of the application. It is a QObject subclass that orchestrates all voice assistant operations and communicates with the UI through Qt signals.

PyQt Signals emitted:

- transcript_added(role, text) - when a user or assistant message should appear
- status_changed(text) - status updates (Listening, Thinking, Speaking, Ready, Idle)
- listening_changed(bool) - toggle listening state in UI
- response_ready(text) - full response text for external consumers
- error_occurred(text) - error messages for UI display

Core methods:

- start_listening() - spawns a daemon thread running the continuous listening loop
- stop_listening() - sets stop event to break the listening loop
- stop_speaking() - terminates the TTS subprocess to stop speech immediately
- submit_text(text) - handles typed text input on a background thread
- _listen_loop() - continuous loop: listen -> recognize -> handle -> repeat
- _handle_command(command) - routes command through TaskRouter or LLM, then speaks reply

Threading model: The listening loop runs on a dedicated daemon thread. Each voice command spawns a separate handler thread to avoid blocking the listener. An Event-based synchronization mechanism ensures commands complete (including speech) before the listener resumes.

4.4 modules/llm_engine.py - LLM Integration

Provides the LLMEngine class that connects to either Groq or OpenAI APIs for generating intelligent conversational responses. The engine maintains conversation context by loading prior messages from the memory system.

How it works:

- Builds a message array: system prompt + conversation history + current user prompt
- Creates the appropriate API client based on LLM_PROVIDER setting
- Sends a chat completion request with temperature=0.5 and max_tokens=450
- Saves both user prompt and assistant response to conversation memory
- Handles API errors, missing keys, and empty responses gracefully

System prompt: 'You are Echo-Mind, a modern desktop AI voice assistant. Be concise, helpful, and safe. If a local task already handled the request, do not invent extra actions.'

4.5 modules/speech_to_text.py - Speech Recognition

Wraps the SpeechRecognition library to provide microphone-based voice input. The SpeechToTextService class captures audio from the microphone and converts it to text using Google's free Speech Recognition API.

Process flow:

- Opens the microphone and adjusts for ambient noise (0.35s calibration)
- Listens for speech with configurable timeout and phrase length limits
- Sends audio to Google Speech Recognition API for transcription
- Returns a SpeechResult dataclass containing either text or an error message

Error handling covers:

- No speech detected (timeout)
- Microphone not available (OSError)
- Unrecognizable audio
- Network/service unavailability
- Missing Python module (setuptools on Python 3.12+)

4.6 modules/text_to_speech.py - Text-to-Speech Service

Manages text-to-speech output using a subprocess-based architecture. Instead of running pyttsx3 directly in the main process (which causes thread-safety issues), it spawns a persistent worker subprocess (tts_worker.py) that handles all speech synthesis.

Architecture:

- On initialization, starts a subprocess running tts_worker.py
- speak_async(text, on_done) sends text via stdin pipe to the worker
- The worker speaks the text and writes 'DONE' to stdout when finished
- A monitoring thread waits for 'DONE' and calls the on_done callback
- stop() terminates the worker process (instantly stopping speech) and restarts it

This design ensures the stop button works reliably - terminating a process is guaranteed to stop all audio output immediately, unlike trying to interrupt pyttsx3 across threads.

4.7 modules/tts_worker.py - TTS Subprocess Worker

A standalone Python script that runs as a child process for speech synthesis. It initializes pyttsx3 once on startup, selects a female voice (Microsoft Zira or similar), and then enters a read-speak loop.

Lifecycle:

- Receives voice rate as a command-line argument
- Initializes pyttsx3 engine and selects female voice
- Writes 'READY' to stdout to signal successful initialization
- Reads text lines from stdin, speaks each one, writes 'DONE' after each
- Runs until the parent process terminates it

4.8 modules/task_router.py - Command Router

The TaskRouter uses regex pattern matching to detect and dispatch specific voice commands to their appropriate handlers. If no pattern matches, the command falls through to the LLM for a conversational response.

Supported command categories:

- YouTube: 'open youtube', 'play [song] on youtube', 'search youtube for [query]'
- Google: 'open google', 'google [query]', 'search google for [query]'
- Screenshots: 'take screenshot', 'capture screenshot'
- App launching: 'open [app]', 'launch [app]', 'start [app]'
- Web search: 'search for [topic]', 'search [topic]'
- Wikipedia: 'wikipedia [topic]', 'wiki [topic]'

Returns a RouteResult dataclass with a 'handled' flag and response text. The assistant controller checks this flag to decide whether to forward the command to the LLM.

4.9 modules/system_control.py - System Automation

Provides functions for OS-level automation including application launching, browser operations, and screenshot capture. Designed for Windows with appropriate subprocess commands.

Functions:

- open_application(target) - launches apps via APP_ALIASES dictionary (Chrome, Notepad, Calc, VS Code, etc.)
- open_url(target) - opens known sites or arbitrary URLs in the default browser
- open_youtube(query) - opens YouTube homepage or search results
- play_on_youtube(query) - scrapes YouTube search results to find and play the first video directly
- open_google(query) - opens Google homepage or search results
- search_web(query) - performs a Google search via browser
- take_screenshot() - captures full screen using PyAutoGUI, saves as timestamped PNG

The module maintains two lookup dictionaries: APP_ALIASES maps application names to their subprocess commands, and KNOWN_SITES maps site names to URLs.

4.10 modules/web_tasks.py - Wikipedia & Web Tasks

Handles Wikipedia topic lookups with intelligent query processing. Extracts the topic from natural language queries, resolves common abbreviations, and fetches concise summaries.

Processing pipeline:

- _extract_topic() - strips prefixes like 'what is', 'who is', 'tell me about'
- _normalize_topic() - resolves aliases (AI -> artificial intelligence, ML -> machine learning)
- _resolve_topic() - handles acronyms by searching Wikipedia and matching results
- search_wikipedia() - fetches a 2-sentence summary with disambiguation handling

4.11 ui/main_window.py - Desktop GUI

Implements the full desktop interface using PyQt6. The window is 1180x760 pixels with a modern dark theme featuring glassmorphism effects, teal accent colors, and smooth animations.

Widget classes:

- WaveformWidget - custom-painted animated waveform with 22 bars, teal when active, gray when idle
- SettingsDialog - modal dialog for entering and saving API keys to .env file
- EchoMindWindow - main application window with two-panel layout

Left panel (60% width):

- Brand header with 'ECHO-MIND' title and Settings button
- Animated waveform visualization
- Status chip (Ready / Listening / Thinking / Speaking / Idle)
- Start Listening button and Stop Speaking button side by side
- Scrollable chat transcript with styled message bubbles
- Text input composer with Send button

Right panel (40% width):

- Mission Control overview with 4 info cards
- Quick Action buttons (YouTube, Play on YouTube, Screenshot, Wikipedia)
- Clear Memory button

Styling: Comprehensive QSS stylesheet with dark navy/teal gradients, glassmorphism panels with semi-transparent backgrounds, rounded corners (30px), custom scrollbars, and distinct colors for user (blue) and assistant (teal) message bubbles.

4.12 utils/helpers.py - Utility Functions

Provides thread-safe utility functions for conversation memory persistence and environment variable management.

Memory functions:

- load_memory() - reads conversation history from JSON file with validation
- save_memory(data) - writes validated messages, enforcing MAX_MEMORY limit
- append_memory(role, content) - adds a single message to conversation history
- clear_memory() - resets conversation history to empty
- validate_message() - ensures messages have valid role and non-empty content

Environment functions:

- upsert_env_value(key, value) - updates or inserts key-value pairs in .env file

All memory operations use a threading Lock (MEMORY_LOCK) to prevent race conditions when multiple threads access the conversation history simultaneously.

5. Technology Stack & Dependencies

PyQt6 6.7.1	Desktop GUI framework providing widgets, layouts, signals/slots, and styling
SpeechRecognition 3.10.4	Speech-to-text library supporting Google, Sphinx, and other APIs
PyAudio 0.2.14	Cross-platform audio I/O for microphone access
pyttsx3 2.90	Offline text-to-speech engine (uses Windows SAPI5)
groq 0.9.0	Python client for Groq API (fast LLM inference with Llama models)
openai 1.51.2	Python client for OpenAI API (GPT models)
wikipedia 1.4.0	Python wrapper for the Wikipedia API
requests 2.32.3	HTTP client library for web requests
PyAutoGUI 0.9.54	Cross-platform GUI automation for screenshots
python-dotenv 1.0.1	Loads environment variables from .env files
psutil 6.0.0	System and process utilities
Pillow 10.3+	Image processing library for screenshot handling
httpx 0.27.2	Modern async HTTP client (required by Groq SDK)

6. Key Design Patterns

Singleton Pattern

TTS, STT, LLM, and TaskRouter services are instantiated once at module level as private singletons. Module-level convenience functions provide a clean public API.

Signal/Slot Pattern (Observer)

PyQt6 signals decouple the backend controller from the UI. The controller emits signals (status_changed, transcript_added, etc.) and the UI connects slots to update widgets. This ensures thread-safe UI updates since signals can cross thread boundaries.

Command Pattern

The TaskRouter implements command pattern matching using regex. Each command category has dedicated handler functions in system_control.py and web_tasks.py.

Subprocess Isolation

Text-to-speech runs in a separate subprocess to avoid thread-safety issues with pyttsx3. This enables reliable stop functionality through process termination.

Graceful Degradation

The app works even without a microphone, speakers, or API keys. Each service checks availability and returns informative error messages rather than crashing.

Thread-safe Data Access

Conversation memory uses a threading Lock to prevent concurrent file access corruption. All memory read/write operations are serialized through this lock.

7. Data Flow

Voice Input Flow

- 1. User clicks 'Start Listening' -> AssistantController.start_listening()
- 2. Listening loop starts on daemon thread, status = 'Listening'
- 3. SpeechToTextService.listen_once() captures audio from microphone
- 4. Audio sent to Google Speech API for transcription
- 5. Recognized text passed to _handle_command() on a new thread
- 6. TaskRouter.route() checks for pattern matches (YouTube, Google, etc.)
- 7. If matched: execute handler, get response. If not: forward to LLMEngine.ask()
- 8. LLM builds message history + prompt, calls Groq/OpenAI API
- 9. Response displayed in transcript via transcript_added signal
- 10. TTS subprocess speaks the response, status = 'Speaking'
- 11. Speech completes, status = 'Ready', listening loop resumes

Text Input Flow

- 1. User types message in input box and clicks 'Send'
- 2. submit_text() spawns handler thread with the typed text
- 3. Same processing pipeline as voice input (steps 6-11 above)

Stop Speaking Flow

- 1. User clicks 'Stop Speaking' button (enabled when status = 'Speaking')
- 2. AssistantController.stop_speaking() called
- 3. TextToSpeechService.stop() terminates the TTS worker subprocess
- 4. Speech stops immediately, new worker process starts for next use
- 5. Status changes to 'Ready'

8. Configuration Reference

All settings are defined in the .env file in the project root:

Variable	Default	Description
ASSISTANT_NAME	Echo-Mind	Display name of the assistant
USER_NAME	User	Display name of the user
DEBUG_MODE	false	Enable debug logging
VOICE_RATE	175	Speech rate in words per minute
LISTEN_TIMEOUT	6	Seconds to wait for speech before timeout
LISTEN_PHRASE_LIMIT	8	Max seconds for a single phrase
LISTEN_RETRY_DELAY_MS	600	Delay between listen retries (ms)
LLM_PROVIDER	groq	LLM backend: 'groq' or 'openai'
GROQ_API_KEY	(empty)	API key for Groq
GROQ_MODEL	llama-3.3-70b-versatile	Groq model identifier
OPENAI_API_KEY	(empty)	API key for OpenAI
OPENAI_MODEL	gpt-4o-mini	OpenAI model identifier
MAX_MEMORY	20	Max conversation messages to retain
SCREENSHOT_DIR	screenshots	Directory for saved screenshots
STT_ENGINE	google	Speech-to-text engine
TTS_ENGINE	pyttsx3	Text-to-speech engine

9. Setup & Usage Instructions

Prerequisites

- Python 3.10 or higher
- Windows OS (for SAPI5 text-to-speech and app launching)
- Working microphone for voice input
- Internet connection for speech recognition and LLM API calls
- Groq or OpenAI API key

Installation Steps

1. Clone or download the project to your local machine

2. Open a terminal in the voice-assistant directory

3. Create a virtual environment: python -m venv venv

4. Activate the environment: venv\Scripts\activate

5. Install dependencies: pip install -r requirements.txt

6. Create a .env file and add your API key:

   GROQ_API_KEY=your_api_key_here

7. Run the application: python main.py

Using Echo-Mind

- Click 'Start Listening' to activate voice input mode
- Speak your command clearly into the microphone
- Or type a message in the text box and click 'Send'
- Use Quick Action buttons for common tasks
- Click 'Stop Speaking' to interrupt the assistant's voice response
- Say 'stop' or 'exit' to end the listening session
- Click 'Settings' to update API keys
- Click 'Clear Memory' to reset conversation history